Multi-Scale Dynamic Convolutional Network for Knowledge Graph Embedding

Zhaoli Zhang, *Member, IEEE*, Zhifei Li, Hai Liu, *Member, IEEE*, and Neal N. Xiong, *Senior Member, IEEE*

Abstract—Knowledge graphs are large graph-structured knowledge bases with incomplete or partial information. Numerous studies have focused on knowledge graph embedding to identify the embedded representation of entities and relations, thereby predicting missing relations between entities. Previous embedding models primarily regard (subject entity, relation, and object entity) triplet as translational distance or semantic matching in vector space. However, these models only learn a few expressive features and hard to handle complex relations, i.e., 1-to-N, N-to-1, and N-to-N, in knowledge graphs. To overcome these issues, we introduce a multi-scale dynamic convolutional network (M-DCN) model for knowledge graph embedding. This model features topnotch performance and an ability to generate richer and more expressive feature embeddings than its counterparts. The subject entity and relation embeddings in M-DCN are composed in an alternating pattern in the input layer, which helps extract additional feature interactions and increase the expressiveness. Multi-scale filters are generated in the convolution layer to learn different characteristics among input embeddings. Specifically, the weights of these filters are dynamically related to each relation to model complex relations. The performance of M-DCN on the five benchmark datasets is tested via experiments. Results show that the model can effectively handle complex relations and achieve state-of-the-art link prediction results on most evaluation metrics.

Index Terms—Knowledge graphs, knowledge graph embedding, complex relations, link prediction, convolutional network

1 Introduction

Recently, knowledge graphs (KGs) have received significant attention due to their widespread application in various artificial intelligence tasks, including recommendation system [1], [2], [3], question answering [4], [5], and dialogue systems [6]. Numerous types of KGs have been created, such as Freebase [7], WordNet [8], and YAGO [9], which are used to store structural information of human knowledge. KGs mainly consist of two elements for storing knowledge. Concrete and abstract concepts are represented as entities, and the relationships between entities are represented as relations. These elements are described as (subject entity, relation, object entity). For example, we know that Tom Cruise was born in New York, and this fact can be represented with the triplet form (*Tom Cruise*, *Birth_in*, *New York*).

One of the main challenges with existing KGs is their incompleteness and partialness. Many of the relations between entities in KGs are missing or incorrect. Thus, KGs have gained massive attention in the field of link prediction. Existing link prediction methods are known as knowledge graph embedding (KGE) [10], which aims to learn the embedding representation of the entities and relations. Extant embedding models can be roughly categorized into translational distance [11], [12], [13], [14] and semantic matching models [15], [16], [17]. These models are simple, fast, and can achieve outstanding results. However, they learn less expressive features in comparison with deep and multi-layer model architecture, which potentially limits their performance.

With its considerable success in natural language processing [18] and computer vision [19], convolutional neural network (CNN) has become a meaningful algorithm tool in many fields. Several CNN-based KGE models [20], [21], [22] have been proposed recently and have reported state-of-the-art link prediction results on several established datasets. These models use multiple layers of nonlinear features to compute the potential link between entities and relations, thereby generating expressive feature embeddings. Furthermore, they have an efficient and fast-to-compute parameter due to optimized GPU implementations.

However, KGE continues to encounter many challenges. Complex relations [11], such as 1-to-N, N-to-1, and N-to-N, exist in KGs. As shown in Fig. 1, as an example of complex relations, 1-to-N means that the subject entity *Tom Cruise* links with multiple object entities *Fighter pilot*, *Agent*, and *Future Soldier* in the relation *Play_as*. Moreover, existing models exhibit poor performance when dealing with complex relations [14]. In the meanwhile, the architecture of existing CNN-based KGE models is still underdeveloped. The only way to increase their expressiveness is to increase embedding size, which leads to a total increase of embedding parameters and does not scale to larger KGs.

- *Zhaoli Zhang and Zhifei Li are with the National Engineering Research Center for E-Learning, Central China Normal University, Wuhan, Hubei 430079, China. E-mail: zl.zhang@ccnu.edu.cn, zhifei1993@mails.ccnu.edu.cn.*
- *Hai Liu is with the National Engineering Research Center for E-Learning, Central China Normal University, Wuhan, Hubei 430079, China. E-mail: hailiu0204@ccnu.edu.cn.*
- *Neal N. Xiong is with the National Engineering Laboratory for Educational Big Data, Central China Normal University, Wuhan, Hubei 430079, China. E-mail: 18766362@qq.com.*

Manuscript received 29 July 2019; revised 24 Mar. 2020; accepted 25 June 2020. Date of publication 30 June 2020; date of current version 1 Apr. 2022. (Corresponding author: Zhifei Li.) Recommended for acceptance by P. Cui. Digital Object Identifier no. 10.1109/TKDE.2020.3005952

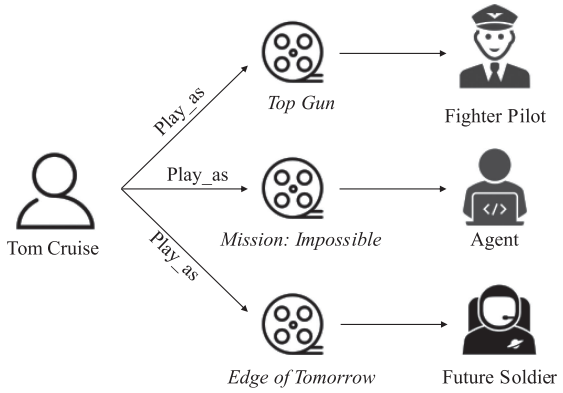


Fig. 1. Illustration of 1-to-N relation in KGs. The subject entity in a relation links with multiple object entities. For example, the movie star *Tom Cruise* has different roles for a relation *play_as*.

To build a high-expressive link prediction framework and effectively deal with the complex relations modeling in KGs, we propose the multi-scale dynamic convolutional network (M-DCN) model for KGE. First, in the input layer, M-DCN reshapes and concatenates the subject entity and relation embeddings in an alternating pattern. Thus, the convolution layer can extract additional feature interactions. Furthermore, in the convolution layer, M-DCN generates multi-scale convolution filters to learn different characteristics between the input embeddings to output feature maps. To model complex relations, the weights of these filters are generated dynamically related to each relation that an entity can have different output feature by each relation. Finally, the tensors of feature maps are vectorized and mapped into the embedding dimension and computed with the object entity vector via an inner product to return the possibility of a triplet. In the experiment, we evaluate M-DCN against several previously proposed link prediction models. The results show that our model can effectively handle complex relations and achieve state-of-the-art results on most evaluation metrics.

In summary, the main contributions of this paper are as follows:

1) A novel CNN-based model M-DCN is proposed to model complex relations in KGs. M-DCN generates the weights of dynamic convolution filters related to each relation. It allows an entity to generate special feature representations under the convolution of different relations. Results show that M-DCN achieves significant improvements in complex relation modeling in comparison with existing baselines.
2) To our best of knowledge, we are the first to utilize multi-scale processing to optimize quality in the convolution layer for KGE. M-DCN generates various sizes of convolution filters to learn different characteristics between the embeddings of entity and relation. In comparison with shallow architectures, M-DCN has a considerable performance improvement at an acceptable increase in computational requirements.
3) Experiments on five representative benchmark datasets are conducted to validate the effectiveness of the proposed method. Results show that M-DCN achieves significant and consistent improvements for most evaluation metrics on link prediction and triplet classification tasks. Thus, the model is generalizable and effective.

The rest of this paper is organized as follows. In Section 2, we briefly review and analyze the related works about KGE. The research problem is formally defined, and the details of the M-DCN are introduced in Section 3. Experimental results of the proposed method on five public databases are presented in Section 4. Finally, the conclusions and the recommendations for future work are discussed in Section 5.

2 Related Work

Predicting missing links with KGE models has been extensively investigated in recent years. Existing KGE models can be roughly categorized into *translational distance models* and *semantic matching models*. With the development of deep neural networks in various fields, several *convolutional network models* have been proposed for KGE.

2.1 Translational Distance Models

Inspired by the skip-gram model [23] in word embeddings, the first translational distance model TransE [11] is proposed. For a triplet (e_s, r, e_o), the model attempts to regard the vector of a relation r as a translational distance between the vectors of subject entity e_s and object entity e_o. The bold letters $\mathbf{e}_s$, $\mathbf{r}$, and $\mathbf{e}_o$ denote the vectors of e_s, r, and e_o, respectively. TransE wants that $\mathbf{e}_s + \mathbf{r} \approx \mathbf{e}_o$ when the triplet holds and optimizes the following loss function:

$$\mathcal{L} = \sum_{(e_s,r,e_o)\cup(e'_s,r,e'_o)} max\left(0, \|\mathbf{e}_s + \mathbf{r} - \mathbf{e}_o\|_p - \|\mathbf{e}'_s + \mathbf{r} - \mathbf{e}'_o\|_p + \gamma\right),$$

where p is the L_1 or L_2 norm and $max(0, x)$ returns the larger value between 0 and x. The γ denotes the margin which is a hyperparameter. The negative triplet (e'_s, r, e'_o) is obtained by replacing the subject or object entities. TransE is simple, fast, and exhibits outstanding performance in large-scale KGs. However, it applies well on 1-to-1 relations but not on complex relations.

To address this issue, TransH [12] supposes that each entity should have a different characteristic with specific relation, and it projects subject and object entities into the specific hyperplane for each relation. TransR [13] models the entities and relations in separate spaces. It projects the embeddings by utilizing a projection matrix related to the relation. TransD [14] further learns two projecting matrices to project respectively. Other similar works, including TransG [24], TorusE [25], [26], ManifoldE [27], SSE [28], and RotatE [29] etc. Several methods recently attempted to improve the performance for complex relations by incorporating additional information including relation paths [30], [31], [32], textual descriptions [33], [34], [35], and temporal information [36] as well as logical rules [37], [38]. However, translational distance models learn less expressive features than deep, multi-layer models, which lead to weak improvements for complex relations modeling.

2.2 Semantic Matching Models

Semantic matching models measure triplet (e_s, r, e_o) plausibility by matching latent semantics in the vector space,

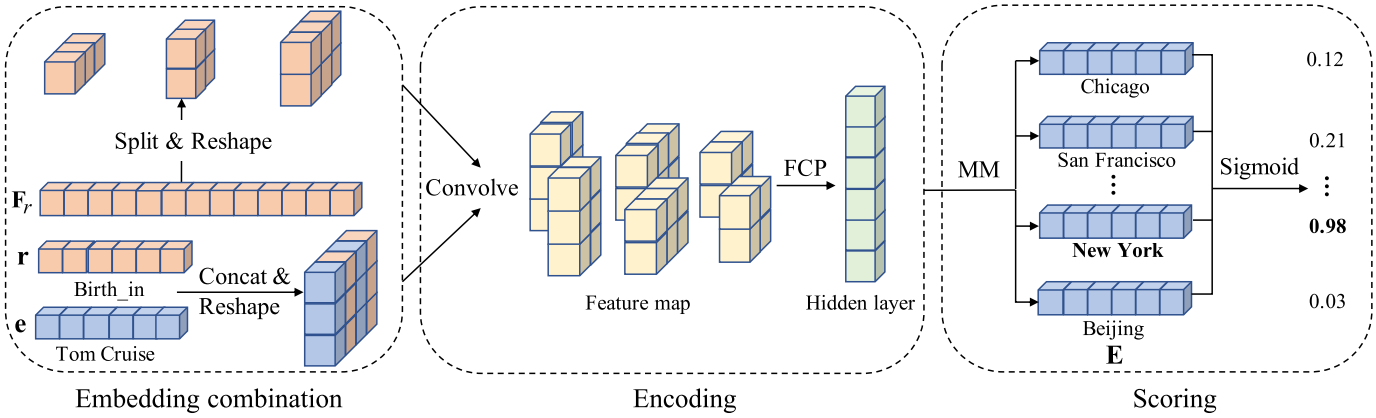


Fig. 2. End-to-end architecture of M-DCN model. Orange and blue cubes represent the embeddings of the subject entity and relation. The yellow cube represents the feature map matrix convolved by filters (orange cube), which are split and reshaped from a relation-specific vector. The green cube represents the hidden layer vectors operated by fully connected projection (FCP). MM denotes the matrix multiplication with all object embeddings. The sigmoid function is used to predict the correct triplet.

where RESCAL [15] is a representative semantic matching model. It contains a bilinear product $\mathbf{e}_s^\top \mathbf{M}_r \mathbf{e}_o$ between the vectors of the subject entity and object entity and a dense matrix for relation, where $\mathbf{e}_s, \mathbf{e}_o \in \mathbb{R}^k$, and $\mathbf{M}_r \in \mathbb{R}^{k \times k}$ represent the matrix associated with the relation. Based on RESCAL, some works [39], [40] consider relational type-constraints and significantly improves link prediction quality. The relational type-constraints explicitly define the logic of relations by excluding entities from the subject or object.

To reduce parameters and eliminate redundancy, DistMult [16] utilize an easy-to-train diagonal matrix to replace the matrices representing relations, but (e_s, r, e_o) and (e_o, r, e_s) exhibit the same results. To tackle this issue, ComplEx [17] extends DistMult to the complex space and model asymmetric relations by taking the conjugate of the object entity embedding. HolE [41] utilizes the circular correlation of embeddings to generate expressive and scalable compositional representations. ANALOGY [42] further models the analogical properties of entities and relations and requires relations linear maps to be normal and mutually commutative. However, semantic matching models have more redundancy than translational distance models. Hence, they are prone to overfitting, which can be a problem if numerous entities and relations exist in a knowledge graph. Accordingly, high-dimensional space is essential to embed and completely differentiate these entities and relations.

2.3 Convolutional Network Models

Recent research has raised interest in applying a convolutional network to link prediction. R-GCN [20] is a generalization of graph convolutional network (GCN) [43] developed for dealing with highly multi-relational data. It utilizes a convolution operator to capture locality information in KGs. However, the framework of GCN is limited to undirected graphs, whereas KGs are naturally directed. ConvE [21] uses 2D convolution over embeddings to predict missing links. It is composed of a convolution, projection, and inner product layer for the final predictions. ConvKB [22] applies the convolutional network to explore the global links among the embeddings of the entity and relation in the same dimensional space. It achieves state-of-the-art link prediction results.

Convolutional network models are expressive and efficient among the three categories. They can capture many kinds of potential features in entities and relations. Therefore, in this paper, we propose a high-efficiency CNN-based model for learning knowledge graph embedding. For complex relations modeling, we intend to extract relation-specific features information from the entity and relation embeddings. In comparison with shallow architectures, the proposed model can significantly improve the performance for link prediction at a modest increase of computational requirements.

3 Methodology

The notations and definitions used in the rest of the paper are first introduced, while the procedure of the link prediction task is explained in this section. The outline of the M-DCN model is then presented in detail. Finally, the training and optimization method for the model is given. The framework of M-DCN is illustrated in Fig. 2.

3.1 Problem Formulation

First, a knowledge graph can be expressed as $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$, where $\mathcal{E} = \{e_1, e_2, \ldots, e_{|\mathcal{E}|}\}$ denotes the set of entities, and $\mathcal{R} = \{r_1, r_2, \ldots, r_{|\mathcal{R}|}\}$ denotes the set of relations. $|\mathcal{E}|$ and $|\mathcal{R}|$ denote the number of entities and relations. $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ denotes the set of triplets and a triplet can be expressed as (e_s, r, e_o). The bold letters $\mathbf{e}_s, \mathbf{r}, \mathbf{e}_o \in \mathbb{R}^k$ denote the k-dimension embeddings of e_s, r and e_o, respectively. Link prediction in KGs aims to predict another entity that given an entity with a specific relation, i.e., predicting object entity e_o given subject entity and relation (e_s, r), denoted as (e_s, r, ?).

Second, to measure the salience of a candidate triplet, the general methodology is to define a score function $\varphi(e_s, r, e_o) \in \mathbb{R}$ for the triplet. The goal of the optimization is usually to score true triplet higher or lower than the corrupted false triplets. Accordingly, CNN-based models for link prediction can be seen as multi-layer neural networks consisting of an encoding component and a scoring component. For an input triplet (e_s, r, e_o), the encoding component extracts a feature vector about the embedding representations of e_s and r. In the scoring component, the feature vector and embedding of eo are scored by the score function $\varphi(e_s, r, e_o)$.

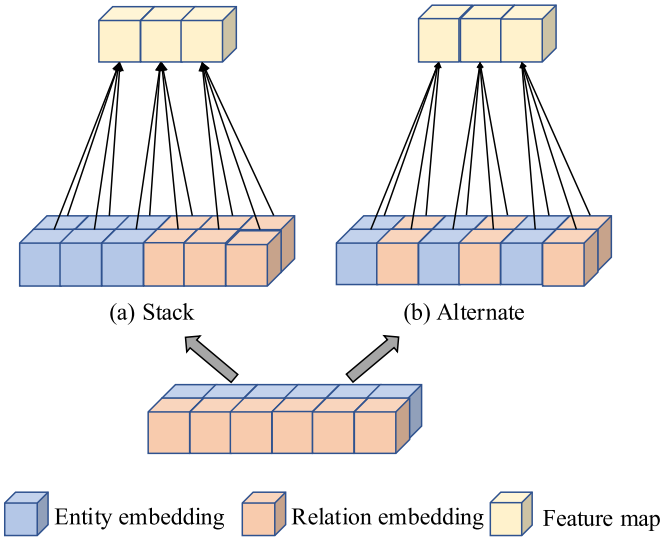


Fig. 3. Two types of input composition for embeddings, (a) *Stack* and (b) *Alternate*.

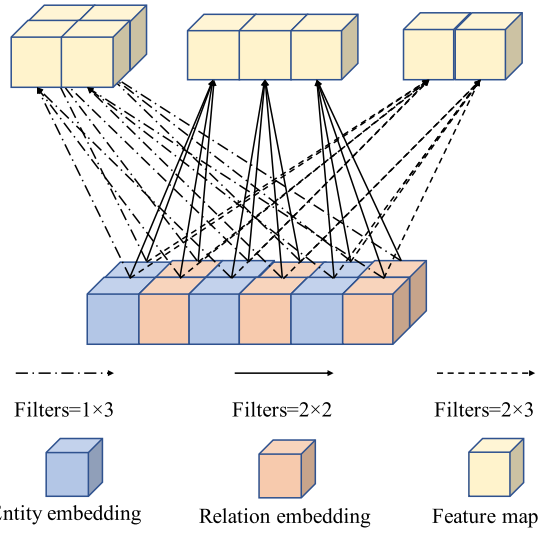


Fig. 4. Multi-scale convolution architecture. The embedding of entity and relation are convolved by multi-scale filters to generate various feature maps.

Third, to capture additional feature interactions from embeddings in the convolutional layer, the general methodology is to transform embeddings $\mathbf{e}_s$ and $\mathbf{r}$ into a matrix $\mathbb{R}^k \times \mathbb{R}^k \rightarrow \mathbb{R}^{k_w \times k_h}$, where $2 \times k = k_w \times k_h$. As shown in Fig. 3, two typical types of input composition for embeddings are defined as follows:

- *Stack* (Fig. 3a) first reshapes each of $\mathbf{e}_s$ and $\mathbf{r}$ into a matrix of shape $\mathbb{R}^{(k_w/2) \times k_h}$, and then stacks them along their height to yield an $\mathbb{R}^{k_w \times k_h}$ matrix.
- *Alternate* (Fig. 3b) first concatenates $\mathbf{e}_s$ and $\mathbf{r}$ to get a two-dimensional $\mathbb{R}^{2 \times k}$ matrix, and then stack k_h rows of the matrix alternately.

3.2 Outline of M-DCN

The M-DCN can be summed up as three stages, namely, embedding representation, encoding component, and scoring component. For triplet (e_s, r, e_o), we first map subject entity e_s, relation r to their distributed embedding representations and generate the vectors of filters related to the relation r. These filters are then reshaped and convolved over embeddings to obtain feature map tensors and hidden layer vectors in the encoding component. Finally, in the scoring component, it is matched with all candidate object embeddings to predict if the triplet is correct or not.

3.2.1 Embedding Representation

For a knowledge graph $\mathcal{G}$, we initialize all entities $\mathcal{E}$ and relations $\mathcal{R}$ as the k-dimensional embeddings. Given an input triplet (e_s, r, e_o), the embedding representation $\mathbf{e}_s, \mathbf{r} \in \mathbb{R}^k$ of subject entity e_s and relation r can be looked up respectively. And it is tended to regard the embedding of e_s, r as a composition and extract their potential characteristics.

The alternate composition method for embeddings is utilized in this paper, as it can model even more feature interactions between entities and relations (with a number of interactions proportional to filter sizes). As shown in Fig. 3b, the $\mathbf{e}_s$ and $\mathbf{r}$ are first concatenated to get a matrix $\mathbf{M}$ as:

$$\mathbf{M} = [\mathbf{e}_s \oplus \mathbf{r}] \in \mathbb{R}^{2 \times k}, \tag{1}$$

where $\oplus$ denotes the concatenate operation. The model's input matrix $\overline{\mathbf{M}}$ can be obtained by reshaping matrix $\mathbf{M}$ as:

$$\overline{\mathbf{M}} = \overline{[\mathbf{e}_s \oplus \mathbf{r}]} \in \mathbb{R}^{k_w \times k_h}, \tag{2}$$

where $\overline{[\cdot]}$ denotes a 2D reshaping and $2 \times k = k_w \times k_h$. In this case, a convolution operation can model additional interactions between $\mathbf{e}_s$ and $\mathbf{r}$. Thus, convolution layer can extract and output additional expressive feature maps.

To model complex relations in a knowledge graph, we intend to generate relation-specific filters $\mathbf{F}_r$ with the motivation that an entity should extract different feature representations in the fact triplets with different relation r. Therefore, the filters can extract relation-specific features from the subject entity embedding. It is more effective to distinguish the difference between entities. Simultaneously, various sizes of filters $[\mathbf{w}_r^1, \mathbf{w}_r^2, \ldots, \mathbf{w}_r^n]$ are generated by $\mathbf{F}_r$ in the convolutional layer to learn multi-scale feature maps. Thus, $\mathbf{F}_r$ can be denoted as:

$$\mathbf{F}_r = \mathrm{vec}[\mathbf{w}_r^1, \mathbf{w}_r^2, \ldots, \mathbf{w}_r^n], \tag{3}$$

where $\mathrm{vec}(x)$ denotes the flatten operation, and the dimension of $\mathbf{F}_r$ depends on the filters sizes. For example, as Fig. 4 illustrates, suppose that three different filters $\mathbf{w}_r^1 \in \mathbb{R}^{1 \times 3}$, $\mathbf{w}_r^2 \in \mathbb{R}^{2 \times 2}$ and $\mathbf{w}_r^3 \in \mathbb{R}^{2 \times 3}$ are generated and operated on the input layer that we can obtain $\mathbf{F}_r \in \mathbb{R}^{1 \times 13}$.

3.2.2 Encoding Component

In the encoding process, a convolution operation is performed on the input layer $\overline{\mathbf{M}}$ by the various filters $[\mathbf{w}_r^1, \mathbf{w}_r^2, \ldots, \mathbf{w}_r^n]$ to generate the feature maps $[\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n]$ as follows:

$$\mathbf{v}_n(i,j) = f\left(\overline{\mathbf{M}} * \mathbf{w}_r^n\right) = f\left(\sum_{a=1}^{k_h}\sum_{b=1}^{k_w} \overline{\mathbf{M}}(i+a, j+b)\mathbf{w}_r^n(a,b)\right), \tag{4}$$

where $*$ denotes the convolution operation, and $f(x)$ is the rectified linear units (ReLU) [19]. Here, we can set the filter

$\mathbf{w}_r^n \in \mathbb{R}^{1\times 2}$ in the convolution operation to utilize the translational property of translational distance models. For instance, we only utilize one filter $\mathbf{w}_r^n = (1, 1)$ during the convolution operation, and the generated feature map $\mathbf{v}_n$ will equal to $\mathbf{e}_s + \mathbf{r}$. The obtained feature maps are then flattened and projected into a k-dimensional space using a linear transformation parametrized by the matrix $\mathbf{W}$ to obtain the hidden layer vector $\mathbf{U} \in \mathbb{R}^k$ as follows:

$$\mathbf{U} = f(\mathrm{vec}[\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n] \cdot \mathbf{W}), \tag{5}$$

the vector $\mathbf{U}$ stands for potential characteristics for the subject entity e_s in the condition of relation r. It is matched with the object entity e_o, with their latent semantics in their vector space representations.

The main advantage of encoding architecture is that it significantly increases the number of interactions at the convolution layer without resulting in computational difficulties. It is inspired by the practical intuition that each entity has different characteristics, which should be processed at various scales and then aggregated. Thus, the next stage can extract features from different scales simultaneously.

3.2.3 Scoring Component

To obtain the score of the triplet (e_s, r, e_o), the hidden layer vectors $\mathbf{U}$ are multiplied with object entity embedding e_o, and the score function $\varphi(e_s, r, e_o)$ of M-DCN can be defined as follows:

$$\varphi(e_s, r, e_o) = \sigma(\mathbf{U} \cdot \mathbf{e}_o + b) \in (0, 1), \tag{6}$$

where b is a bias term, and $\sigma(x) = 1/1 + \exp(-x)$ is the logistic sigmoid function to give a probabilistically interpretable prediction if the triplet (e_s, r, e_o) is correct or not. Based on equations (1)-(6), the score function can also be written as follows:

$$\begin{gathered}\varphi(e_s, r, e_o) = \sigma(f\Big(\mathrm{vec}\Big(f\Big(\overline{[\mathbf{e}_s \oplus \mathbf{r}]} * \mathbf{w}_r^i\Big)\Big) \cdot \mathbf{W}\Big) \cdot \mathbf{e}_o + b\Big), \\ \mathbf{w}_r^i \in \left[\mathbf{w}_r^1, \mathbf{w}_r^2, \ldots, \mathbf{w}_r^n\right]\end{gathered} \tag{7}$$

the parameters of the linear transformation matrix $\mathbf{W}$ and the bias b are shared and independent of the parameters for the entities, relations, and filters.

To minimize the number of convolution operations and speed up computation for our model, the 1-N scoring technique is adopted in the scoring process. Specifically, in contrast to 1-1 scoring, the (e_s, r) is taken as a pair and simultaneously scored it against all entities $\mathcal{E}$. 1-N scoring affords a significant speedup on training and testing time and improves the accuracy in comparison to 1-1 scoring.

3.3 Training Objective

In M-DCN, the observed knowledge graph $\mathcal{G}$ and parameters Θ are intended to maximize the likelihood function $p(\mathcal{G}|\Theta)$, which is given as follows:

$$\max p(\mathcal{G}|\Theta), \tag{8}$$

where Θ represents all parameters in our model, including the embeddings of entities and relations, filters vectors, linear transformation matrix, and bias.

TABLE 1
Real-World KGs Datasets

Datasets	# *Entities*	# *Relations*	# *Triplets*		
			Train	*Valid*	*Test*
FB15k	14,951	1,345	483,142	50,000	59,071
WN18	40,943	18	141,442	5,000	5,000
FB13	75,043	13	316,232	5,908	23,733
WN11	38,696	11	112,581	2,609	10,544
FB15k-237	14,541	237	272,115	17,535	20,466
WN18RR	40,943	11	86,835	3,034	3,134
YAGO3-10	123,182	37	1,079,040	5,000	5,000

In this paper, for the correct triplet (e_s, r, e_o) in , we expect its scoring $\varphi(e_s, r, e_o)$ to be equal to 1, otherwise 0. The likelihood function is defined as the Bernoulli distributions.

$$p(\mathcal{G}|\Theta) = \prod_{(e_s,r,e_o)\in\{\mathcal{T}\cup\mathcal{T}'\}} (\varphi(e_s, r, e_o))^y \, (1 - \varphi(e_s, r, e_o))^{1-y}, \tag{9}$$

in which

$$y = \begin{cases} 1 & \text{for } (e_s, r, e_o) \in \mathcal{T} \\ 0 & \text{for } (e_s, r, e_o) \in \mathcal{T}' \end{cases}, \tag{10}$$

where $\mathcal{T}'$ is a set of incorrect triplets generated by corrupting the correct triplets set $\mathcal{T}$. Based on Equations (8), 9, and (10), the loss function for M-DCN can be defined as follows:

$$\begin{aligned}\min \mathcal{L} &= -\log p(\mathcal{G}|\Theta) \\ &= -\sum_{(e_s,r,e_o)\in\{\mathcal{T}\cup\mathcal{T}'\}} \Big(y \log \varphi(e_s, r, e_o) \\ &\quad + (1 - y) \log (1 - \varphi(e_s, r, e_o))\Big).\end{aligned} \tag{11}$$

The normalization initialization technique [44] is used to initialize all parameters. We regularize the model by using dropout [45] in several stages, including the feature maps after the convolution operation and the hidden layer after the fully connected projection. The batch normalization [46] is adopted after each layer to stabilize, regularize, and increase the rate of convergence. We also utilize the label smoothing technique to lessen overfitting and improve generalization [47]. The Adam optimizer [48] is taken to minimize the loss function, which is a fast and computationally efficient tool for gradient-based optimization.

4 Experimentals and Discussion

4.1 Datasets

We conduct extensive experiments for link prediction task on the following benchmark datasets: FB15k, WN18, FB13, WN11, FB15k-237, WN18RR, and YAGO3-10. The details of these datasets are shown in Table 1. As noted by [21], [49], many reversible relations exist in WN18 and FB15k. Thus, predicting most test triplets is easy. Thus, WN18RR and FB15k237 are created that reversible relations are removed.

1) *FB15k* [11] was extracted from Freebase [7], which contains approximately 14,951 entities and 1,345 relations. It is a large knowledge base about the real

TABLE 2
Results of the Link Prediction by MRR and Hits@ k on WN18 and FB15k Datasets

Models	WN18				FB15k			
	MRR	Hits			MRR	Hits		
		@1	@3	@10		@1	@3	@10
TransE [11]	0.454	0.089	0.823	0.934	0.380	0.231	0.472	0.641
TransR [13]	0.605	0.335	0.876	0.940	0.346	0.218	0.404	0.582
DistMult [16]	0.822	0.728	0.914	0.936	0.654	0.546	0.733	0.824
ComplEx [17]	0.941	0.936	0.936	0.947	0.692	0.599	0.759	0.840
R-GCN [20]	0.814	0.686	0.928	0.955	0.696	0.601	0.760	0.842
ConvE [21]	0.943	0.935	0.946	0.956	0.657	0.558	0.723	0.831
TorusE [26]	0.947	0.943	0.950	0.954	0.733	0.674	0.771	0.832
RotatE [29]	0.948	0.943	0.952	**0.958**	**0.782**	**0.735**	**0.823**	**0.882**
DCN	0.948	0.944	0.952	0.956	0.757	0.685	0.805	0.870
M-DCN	**0.950**	**0.946**	**0.954**	**0.958**	0.762	0.701	0.820	0.879

world, which describes facts about movies, actors, awards, sports, and sports teams, among others.

2) *WN18* [11] was extracted from WordNet [8], which contains 40,943 entities and 18 relations. Its entities represent the word senses, while its relations define lexical relationships between entities.
3) *FB13* and *WN11* [14] were used for triplet classification tasks. The test sets of FB13 and WN11 contain both correct and incorrect triplets.
4) *FB15k-237* [21] is a subset of FB15k with the reversible relations removed. It contains 14,541 entities with 237 different relations.
5) *WN18RR* [21] is a subset of WN18 with the inverse reversible removed. It contains approximately 40,943 entities with 11 different relations.
6) *YAGO3-10* [21] is a subset of YAGO [9], which contains approximately 123,182 entities with 37 different relations. Each entity has a minimum of 10 relations. It describes the attributes of peoples, such as profession, gender, and citizenship.

4.2 Evaluation Metrics

For a knowledge graph $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$, let $\mathcal{T}_{test} = \{x_1, x_2, \ldots, x_{|\mathcal{T}_{test}|}\}$ denote the test set. Following the filtered setting in TransE [11], for the ith test triplet x_i, we generate all its possible incorrect triples $\tilde{x}_i^s \notin \mathcal{T}$ (resp. $\tilde{x}_i^o \notin \mathcal{T}$) obtained by replacing its subject (resp. object) with any other entity in the knowledge graph. We then check if the model acquires a higher score to x_i and a lower score to incorrect triplets. The left and right ranks of the ith test triplet each associated to corrupting either the subject or the object according to the score function $\varphi(e_s, r, e_o)$ in our model, which are defined as follows:

$$\begin{aligned} \text{rank}_i^s &= 1 + \textstyle\sum_{\tilde{x}_i^s \notin \mathcal{T}_{test}} I\left[\varphi(x_i) < \varphi\left(\tilde{x}_i^s\right)\right] \\ \text{rank}_i^o &= 1 + \textstyle\sum_{\tilde{x}_i^o \notin \mathcal{T}_{test}} I\left[\varphi(x_i) < \varphi\left(\tilde{x}_i^o\right)\right], \end{aligned} \tag{12}$$

where $I[P]$ is an indicator function that the condition P is true returns 1, otherwise returns 0. In our experiments, two popular ranking metrics are utilized to measure the accuracy of prediction. It includes Mean Reciprocal Rank (MRR) and Hits@k metrics (for k =1, 3, and 10), which are defined as follows:

$$\text{MRR} : \frac{1}{2|\mathcal{T}_{test}|} \sum_{x_i \in \mathcal{T}_{test}} \left(\frac{1}{\text{rank}_i^s} + \frac{1}{\text{rank}_i^o}\right), \tag{13}$$

and

$$\text{Hits@}k : \frac{1}{2|\mathcal{T}_{test}|} \sum_{x_i \in \mathcal{T}_{test}} I\left[\text{rank}_i^s \le k\right] + I\left[\text{rank}_i^o \le k\right], \tag{14}$$

where MRR is the average inverse rank for all test triplets, and Hits@k is the percentage of ranks lower than or equal to k. In both conditions, higher MRR or Hits@k indicates better performance.

4.3 Experimental Implementation

For the experiments of M-DCN, the hyperparameters were selected via grid search on the validation dataset to obtain the suitable configuration. The hyperparameter ranges for the grid search were as follows: entity and relation embedding size [50, 100, 200, 400] feature map and hidden layer dropout [0.0, 0.1, 0.2, 0.3, 0.4, 0.5], learning rate [0.0005, 0.001, 0.003, 0.005], and label smoothing [0.0, 0.1, 0.2, 0.3], batch size [64, 128, 256]. We investigated the influence of the 1D or 2D convolution layer for our models. Different 1D filter sizes [1×3, 1×5, 1×9, 1×12] and 2D filter sizes [2×2, 3×3, 5×5] were tested in the experiments.

The following combination of parameters works well on all datasets: entity and relation embedding size 200, label smoothing 0.1, batch size 128, filters combination [1×3, 1×9, 3×3]. We set learning rate 0.003, feature map dropout 0.2, hidden layer dropout 0.3 for FB15k, WN18 and YAGO3-10 and learning rate 0.0005, feature map dropout 0.3, hidden layer dropout 0.4 for FB15k-237, WN18RR. Additionally, we explored the effect of using individual filter size and propose the DCN model, which only uses the filter size 3×3. We implemented our models and replicated the DistMult, ComplEx, and ConvE via software library PyTorch [50] on a PC server equipped with NVIDIA GTX 1080Ti.

4.4 Results and Discussion

4.4.1 Overall Results

In this part, the performance of different models are shown on five benchmark datasets, and further analysis are offered. The baseline models for comparison include TransE [11],

TABLE 3
Results of the Link Prediction by MRR and Hits@ k on WN18RR and FB15k-237 Datasets

Models	WN18RR				FB15k-237			
	MRR	Hits			MRR	Hits		
		@1	@3	@10		@1	@3	@10
TransE [11]	0.182	0.027	0.295	0.444	0.257	0.174	0.284	0.420
TransR [13]	0.401	0.345	0.389	0.465	0.263	0.168	0.267	0.428
DistMult [16]	0.430	0.390	0.440	0.490	0.241	0.155	0.263	0.419
ComplEx [17]	0.440	0.410	0.460	0.510	0.247	0.158	0.275	0.428
R-GCN [20]	0.226	0.157	0.269	0.376	0.249	0.151	0.263	0.417
ConvE [21]	0.430	0.400	0.440	0.520	0.325	0.237	0.356	0.501
TorusE [26]	0.453	0.422	0.464	0.512	0.316	0.217	0.335	0.484
RotatE [29]	0.471	0.420	**0.485**	**0.554**	0.336	0.240	0.372	**0.530**
DCN	0.463	0.435	0.481	0.533	0.340	0.244	0.374	0.521
M-DCN	**0.475**	**0.440**	**0.485**	0.540	**0.345**	**0.255**	**0.380**	0.528

TransR [13], TorusE [26] and RotatE [29], DistMult [16], ComplEx [17] and R-GCN [20], and ConvE [21]. The results of RESCAL, DistMult, ComplEx, R-GCN, ConvE, and TorusE were reported in [21], and the others were reported in [26].

The results for link prediction task are shown in Tables 2, 3, and 4. The best score is in **bold** and the second-best score is underlined. The Hits@10 and MRR at the training process on FB15k-237 and WN18RR are depicted in Fig. 5. First, convolutional network models generally have better performance than the previous models. In addition, semantic matching models are easy to train and can quickly model a knowledge graph, as manifested in Fig. 5. However, they are prone to overfitting due to their redundancy, which leads to worse performance than convolutional network models. Moreover, the expressiveness and robustness of the convolutional network for KGE are confirmed. Second, M-DCN achieves new state-of-the-art results for most evaluation metrics. In contrast to the best baseline model on FB15k-237, WN18RR, and YAGO3-10, M-DCN has 0.8, 2.6 and 3.6 percent relative improvement in MRR and 4.8 percent, 6.3 and 6.3 percent relative improvement in Hits@1. It demonstrates the effectiveness and generality of our models. Third, M-DCN has better performance than DCN because it utilizes multi-scale filters to learn different characteristics. Thus, multi-scale convolutional architecture can potentially improve the performance of our model.

TABLE 4
Results of the Link Prediction by MRR and Hits@ k on YAGO3-10 Dataset

Models	YAGO3-10			
	MRR	Hits		
		@1	@3	@10
TransE [11]	0.238	0.212	0.361	0.447
TransR [13]	0.256	0.223	0.356	0.478
DistMult [16]	0.340	0.240	0.380	0.540
ComplEx [17]	0.360	0.260	0.400	0.550
R-GCN [20]	0.235	0.212	0.376	0.421
ConvE [21]	0.440	0.350	0.490	0.620
TorusE [26]	0.481	0.387	0.521	0.640
RotatE [29]	0.487	0.398	0.545	0.667
DCN	0.499	0.418	0.567	0.678
M-DCN	**0.505**	**0.423**	**0.587**	**0.682**

In conclusion, the experimental results prove that M-DCN is a highly expressive KGE model for link prediction and triplet classification. Based on its architecture, our model achieved significant improvements for all evaluation metrics by modeling complex relations in KGs. It can also enhance model effectiveness by using multi-scale filters to learn different characteristics between the embeddings of entity and relation.

4.4.2 Complex Relations Modeling

The performance of different models for complex relations modeling is evaluated on FB15k and WN18 because they contain various relations. The baseline models for comparison include TransE [11], TransD [14], TorusE [25], TransE $_{TC}$ [40], DCN and DistMult [16], ComplEx [17], and ConvE [21]. The results of TransE, TransD, and TorusE were reported by [26], and the others were implemented and reproduced by us.

According to TransE [11], relations in KGs can be classified into 1-to-1, 1-to-N, N-to-1, and N-to-N. For each relation r, it needs to compute the average number of subject (head) entities per object (tail) entity hpt_r and the average number of object entities per subject tph_r. The four relations can be classified as follows:

$$\begin{cases} hpt_r < \eta \text{ and } tph_r < \eta \Rightarrow 1-\text{to}-1 \\ hpt_r < \eta \text{ and } tph_r \geq \eta \Rightarrow 1-\text{to}-\text{N} \\ hpt_r \geq \eta \text{ and } tph_r < \eta \Rightarrow \text{N}-\text{to}-1 \\ hpt_r \geq \eta \text{ and } tph_r \geq \eta \Rightarrow \text{N}-\text{to}-\text{N} \end{cases}, \tag{15}$$

where η=1.5 is the same as TransE [11]. As a result, 353, 305, 380, 307, and 2, 7, 7, 2 relations belong to 1-to-1, 1-to-N, N-to-1, and N-to-N on FB15k and WN18, respectively.

The results of the Hits@10 for each relation category on FB15k are shown in Table 5. The best score is in *bold* and the second-best score is underlined. M-DCN performs better than baselines on the category of predicting object entity. In comparison with the best baseline model, it has 1.0-5.4 percent relative improvement of Hits@10. As for predicting subject entity, M-DCN also achieves outstanding performance apart from N-to-N relation. The results for each relation of MRR on WN18 are shown in Table 6. M-DCN achieves state-of-the-art performance for most relations. Among these relations, 61.1 percent of MRR is greater than or equal to 0.95.

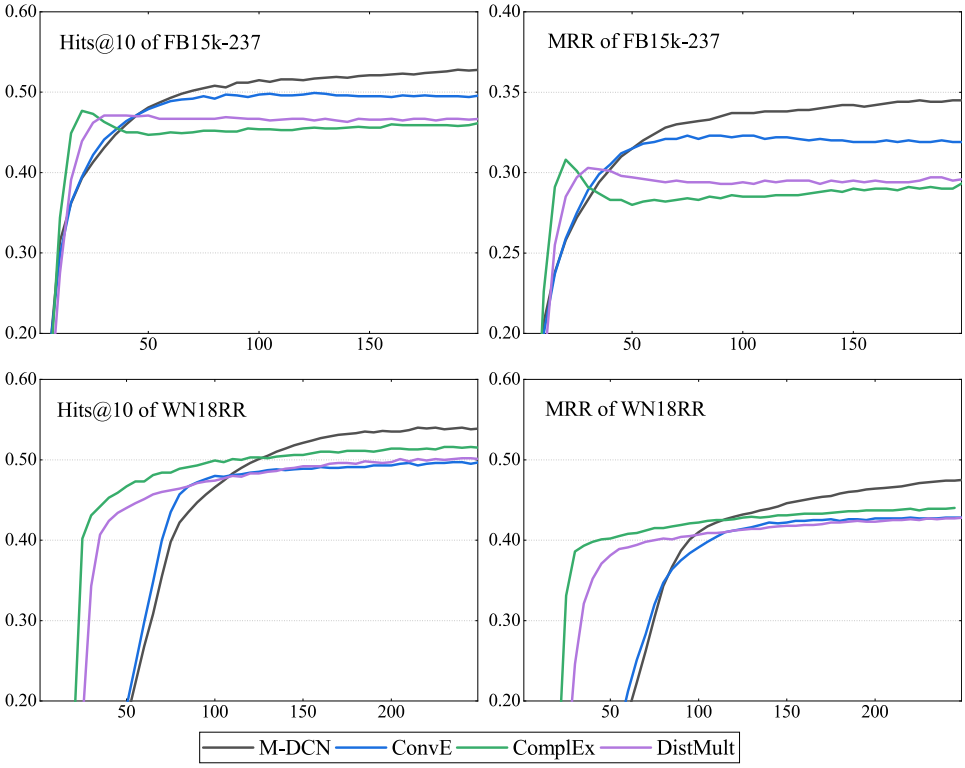


Fig. 5. Training processes compared with Hits@10 and MRR on FB15k-237 and WN18RR datasets.

The results indicate that M-DCN performs well for complex relations modeling on both datasets, which demonstrates the effectiveness of the model. Therefore, the model can capture many kinds of relations in KGs. By generating the weights of convolutional filters specific to each relation, the different output features can be outputted during the convolution process for an entity. Thus, distinguishing between different entities is more effective because it can model various kinds of complex relations.

4.4.3 Triplet Classification Task

Triplet classification consists in verifying whether an unseen triplet fact (e_s, r, e_o) is correct or not. This task can be seen as some sort of completion of the input KGs, which has also been studied extensively in previous works. The results of triplets classification accuracy (%) on WN11, FB13, and FB15k datasets are shown in Table 7. The best score is in *bold* and the second-best score is underlined. We can conclude that DCN and M-DCN outperform any other previous models for triplet classification task. The classification accuracy of M-DCN on FB13 datasets achieves 89.5 percent, which is significantly higher than translational distance models. In comparison with the best baseline model, M-DCN has 1.0, 4.7 and 4.1 percent relative improvement of classification accuracy on three datasets, respectively. Moreover, M-DCN has better performance than DCN because it utilizes multi-scale filters to learn different characteristics. Thus, multi-scale convolutional architecture can potentially improve performance.

4.4.4 Effect of Input Composition

In this part, we empirically test the effectiveness of different input composition types. We evaluate different variants of DCN and M-DCN on FB15k-237 and WN18RR datasets. The results are summarized in Table 8 and the best score is

TABLE 5
Comparison Results of Relation Categories by Hits@10 on FB15k Dataset

Models	Predicting Subject Entity (Hits@10)				Predicting Object Entity (Hits@10)			
	1-to-1	1-to-N	N-to-1	N-to-N	1-to-1	1-to-N	N-to-1	N-to-N
TransE [11]	0.437	0.657	0.182	0.472	0.437	0.197	0.667	0.500
TransD [14]	0.799	0.924	0.433	0.777	0.804	0.529	0.927	0.807
DistMult [16]	0.889	0.954	0.539	0.795	0.884	0.624	0.949	0.879
ComplEx [17]	0.885	0.947	0.404	0.683	0.888	0.511	0.939	0.750
ConvE [21]	0.866	0.959	0.598	0.786	0.852	0.704	0.957	0.858
TorusE [26]	0.775	0.906	0.479	**0.859**	0.773	0.592	0.893	0.890
TransE$_{TC}$ [40]	0.805	0.924	0.442	0.781	0.801	0.517	0.917	0.785
DCN	0.902	0.954	0.624	0.825	0.902	0.715	0.952	0.899
M-DCN	**0.915**	**0.966**	**0.641**	0.843	**0.921**	**0.742**	**0.967**	**0.914**

TABLE 6
Comparison Results of Each Relation by MRR on WN18 Dataset

Relation category	Relation name	Models								
		M-DCN	DCN	TransE$_{TC}$ [40]	TorusE [26]	ConvE [21]	ComplEx [17]	DistMult [16]	TransD [14]	TransE [11]
1-to-1	similar_to	**1.000**	**1.000**	0.735	**1.000**	0.917	**1.000**	0.832	0.724	0.242
	verb_group	0.969	0.962	0.765	**0.974**	0.805	0.936	0.782	0.746	0.283
1-to-N	has_part	**0.946**	0.935	0.784	0.944	0.861	0.933	0.854	0.733	0.417
	hyponym	**0.957**	0.952	0.732	0.956	0.882	0.946	0.861	0.653	0.379
	instance_hyponym	**0.977**	0.971	0.828	0.961	0.957	0.945	0.924	0.726	0.626
	member_meronym	**0.935**	0.930	0.759	0.931	0.816	0.921	0.783	0.678	0.433
	member_of_domain_region	**0.942**	0.934	0.876	0.885	0.923	0.865	0.901	0.872	0.358
	member_of_domain_usage	**0.960**	0.950	0.862	0.917	0.928	0.917	0.912	0.879	0.270
	member_of_domain_topic	0.927	0.921	0.834	**0.944**	0.922	0.924	0.899	0.783	0.502
N-to-1	hypernym	**0.959**	0.950	0.727	0.957	0.891	0.953	0.854	0.652	0.376
	instance_hypernym	**0.968**	0.961	0.811	0.961	0.948	0.965	0.922	0.854	0.680
	member_holonym	**0.950**	0.946	0.734	0.942	0.837	0.946	0.814	0.753	0.438
	part_of	0.942	0.932	0.673	**0.947**	0.881	0.940	0.842	0.684	0.415
	synset_domain_region_of	**0.959**	0.948	0.826	0.919	0.946	0.919	0.918	0.783	0.197
	synset_domain_topic_of	**0.942**	0.935	0.735	0.921	0.890	0.930	0.813	0.693	0.536
	synset_domain_usage_of	0.973	0.964	0.856	0.940	0.960	**1.000**	0.927	0.883	0.182
N-to-N	also_see	**0.643**	0.634	0.447	0.626	0.634	0.603	0.571	0.462	0.257
	derivationally_related_form	**0.956**	0.949	0.612	0.951	0.821	0.946	0.726	0.596	0.362

in *bold*. We can conclude that the performance with *stack* type in both DCN and M-DCN is the worst, and it improves when we replace it with *alternate* type. The reason is that the feature interactions of *stack* type can only be learned at the concatenation part of entities and relations embeddings. For *alternate* type, the convolution operation of 2D filters can extract feature interactions between entities and relations each time. And it is able to model even more interactions between entities and relations with a number of interactions proportional to filter sizes. The same principle can be extended to higher-dimensional embeddings. In conclusion, the observation can prove that *alternate* input composition increases the expressiveness of our model through additional points of interaction between embeddings.

TABLE 7
Results of Triplets Classification Accuracy (%) on WN11, FB13 and FB15k Datasets

Models	Datasets		
	WN11	FB13	FB15k
TransE [11]	75.9	70.9	77.3
TransH [12]	77.7	76.5	74.2
TransR [30]	85.5	82.5	82.1
TransD [14]	85.6	74.7	81.1
DistMult [16]	85.2	82.7	82.5
ComplEx [17]	85.8	83.2	84.2
ConvE [21]	86.1	85.5	85.1
DCN	86.5	89.1	87.8
M-DCN	**87.0**	**89.5**	**88.6**

TABLE 8
Influence of Different Input Composition Choices on WN18RR and FB15k-237 Datasets

Datasets	Metrics	DCN		M-DCN	
		Stack	Alternate	Stack	Alternate
FB15k-237	Hits@10	0.506	0.521	0.515	**0.528**
	MRR	0.317	0.340	0.323	**0.345**
WN18RR	Hits@10	0.514	0.533	0.526	**0.540**
	MRR	0.448	0.463	0.457	**0.475**

4.4.5 Effect of Multi-Scale Convolution

Simply stacking larger convolutional layers is very computationally expensive and cause an increase in model parameters. To make better use of computing resources within the networks and improve model performance, we utilize multi-scale processing to optimize quality in the convolution layer. To explore the influence of multi-scale filter sizes, some comparison experiments on FB15k-237 and WN18RR datasets are expanded to reveal the relevance. The results are shown in Table 9 and the best score is in *bold*. We can

TABLE 9
Influence of Multi-Scale Filter Sizes on WN18RR and FB15k-237 Datasets

Models	Filter sizes	FB15k-237		WN18RR	
		MRR	Hits@10	MRR	Hits@10
DCN	1×3	0.309	0.497	0.449	0.516
	1×5	0.322	0.509	0.457	0.526
	1×9	0.327	0.512	0.459	0.530
	1×12	0.315	0.506	0.454	0.524
	2×2	0.311	0.495	0.454	0.522
	3×3	**0.340**	**0.521**	**0.463**	**0.533**
	5×5	0.313	0.498	0.451	0.520
M-DCN	$\begin{bmatrix} 1 \times 5 \\ 1 \times 9 \\ 2 \times 2 \end{bmatrix}$	0.341	0.524	0.470	0.537
	$\begin{bmatrix} 1 \times 5 \\ 1 \times 9 \\ 3 \times 3 \end{bmatrix}$	**0.345**	**0.528**	**0.475**	**0.540**
	$\begin{bmatrix} 1 \times 5 \\ 1 \times 9 \\ 5 \times 5 \end{bmatrix}$	0.338	0.521	0.472	0.535

TABLE 10
Running Time (s) Comparison for Each Epoch on Four Datasets

Models	Space complexity $\mathcal{O}$	Datasets			
		FB15k	WN18	FB15k-237	WN18RR
ConvE	$(\lvert\mathcal{E}\rvert + \lvert\mathcal{R}\rvert)k$	41.70	57.79	22.86	35.24
DCN	$(\lvert\mathcal{E}\rvert + \lvert\mathcal{R}\rvert)k$	42.41	57.68	22.85	35.33
M-DCN	$(\lvert\mathcal{E}\rvert + \lvert\mathcal{R}\rvert)k$	46.99	60.20	25.28	37.13

find that moderate filter sizes (1×5, 1×9 and 3×3) usually have content results in the convolution operation for DCN. Smaller filter sizes limit the number of interactions and larger filter sizes will lead to overfitting. M-DCN has considerable performance improvement by combining these moderate filters. The results prove that different convolution filters can obtain different feature information of the input embeddings. And combining all the results will get better feature information extraction.

4.4.6 Model Efficiency Evaluation

We compare the running time (s) for each epoch with ConvE [21] using FB15k, WN18, FB15k-237 and WN18RR datasets. The experiment was conducted on a PC server equipped with NVIDIA GTX 1080Ti. Table 10 shows the experiment results. The space complexity of three models are $\mathcal{O}((|\mathcal{E}| + |\mathcal{R}|)k)$, where $|\mathcal{E}|$ and $|\mathcal{R}|$ represent the number of entities and relations respectively and k denotes the k-dimension embeddings. We can find that ConvE and DCN have a similar time consumption for each epoch on four datasets. And M-DCN has a minor increase for time consumption in comparison with the former two models. Where M-DCN adopts multi-scale filters in one convolutional layer to improve model performance instead of simply stacking larger convolutional layers. The results prove that M-DCN has a considerable performance improvement at an acceptable increase in computational requirements.

4.4.7 Parameter Sensitivity Analysis

Dropout Rate Influence Parameter. The dropout technique is utilized to boost the generalization ability of M-DCN. To analyze the effect of dropout rate for our model, we performed several experiments with the dropout rate sampled from 0.0 to 0.5 on four datasets. Fig. 6 shows the Hits@10 results for the influence of different feature map and hidden layer dropout rates on the performance of M-DCN. The results with dropout have achieved better performance than those without dropout. Therefore, the dropout technique can improve model performance effectively. Feature map 0.2 and hidden layer 0.3 achieved the best performance for FB15k and WN18, and feature map 0.3 and hidden layer 0.4 achieved the best performance for FB15k-237 and WN18RR. The possible reason for this finding is that the former two datasets contain reversible relations, which makes predicting easy.

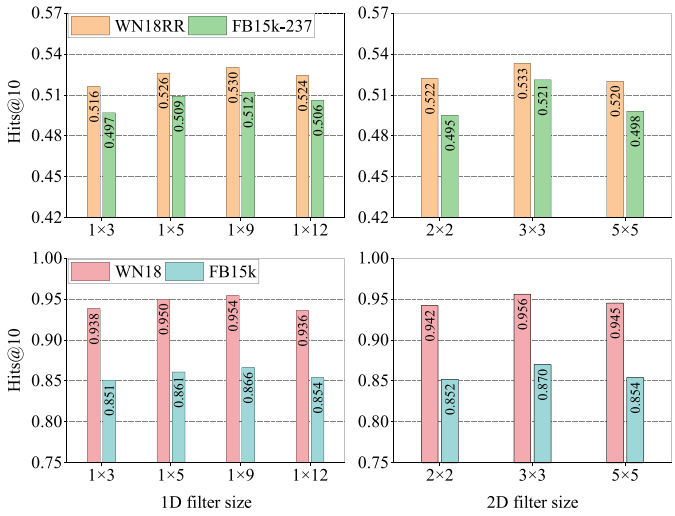


Fig. 7. Performance of Hits@10 with different filter sizes on four datasets.

Filter Sizes Study. To explore the combination of multi-scale filter sizes in the convolution operation, we investigated the influence of 1D and 2D convolutional filter dimension on the performance for our models. The results are shown in Fig. 7. In the left part of the figure, we vary the length of 1D filter from 1×3, 1×5, 1×9 to 1×12, showing that outstanding results can be achieved with filter sizes 1×5 and 1×9, with diminishing results for smaller filter size 1×3 and larger filter size 1×12. The right part of the figure shows results for 2D filter convolved over embeddings, It can be observed that filter size 3×3 achieved the best performance compared with smaller filter size 2×2 or larger filter size 5×5. Therefore, moderate filter sizes usually have content results in the convolution operation on knowledge graph embeddings. The possible reason is that smaller filter sizes limit the number of interactions. Simultaneously, larger filter sizes will lead to overfitting. Therefore, the multi-scale filter sizes 1×5, 1×9, and 3×3 are utilized in M-DCN to convolve the embeddings of entity and relation, which obtains the results of the experiment.

Label Smoothing Influence. Label smoothing is an effective technique for avoiding overfitting when calculating loss

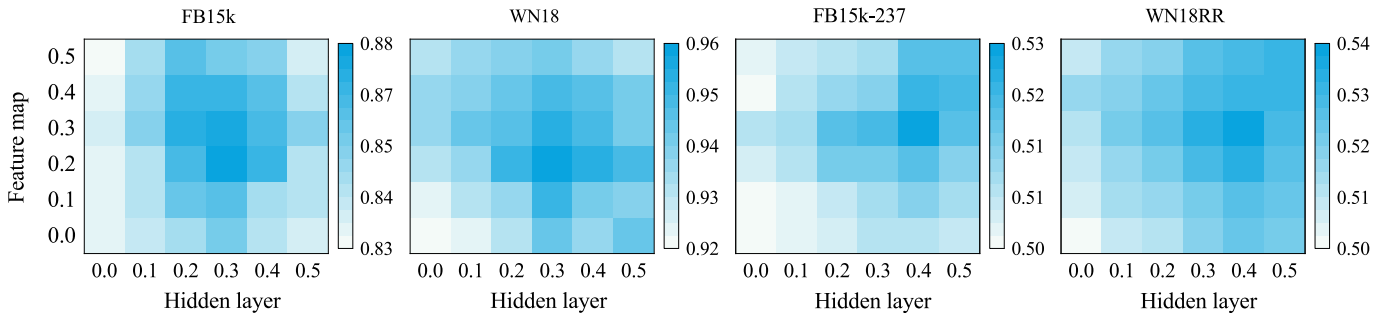


Fig. 6. Performance of Hits@10 with different dropout rate on four datasets.

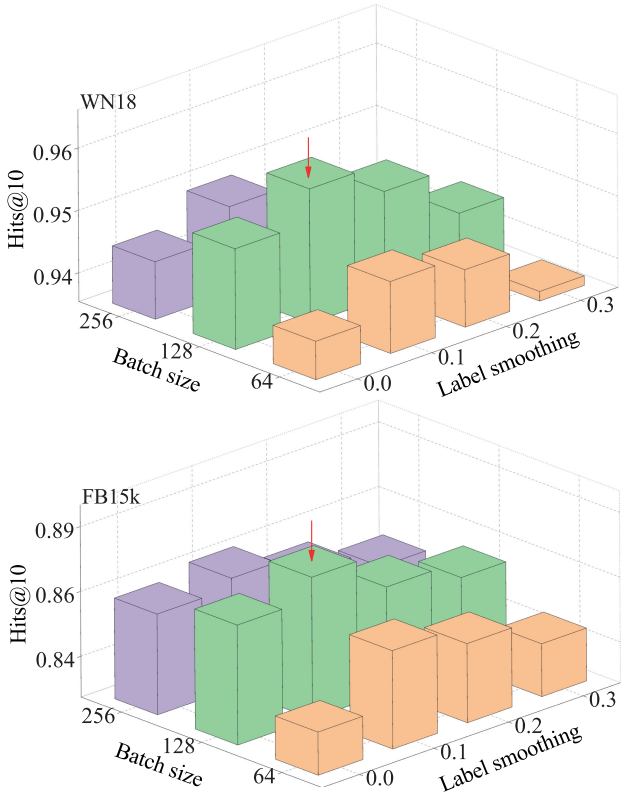


Fig. 8. Performance of Hits@10 with different label smoothing and batch size on FB15k and WN18.

values. This technique is related to the batch size for the link prediction task. Therefore, some comparison experiments on FB15k and WN18 are expanded to reveal the relevance between the model performance and these parameters. The results are shown in Fig. 8. We can observe that our model performs better for Hits@10 on both datasets when the label smoothing value is 0.1. Hence, it confirms that label smoothing can improve the performance of our model to a certain extent. However, increasing when we increase the label smoothing value, it will lead to underfitting. Thus, it is recommended to evaluate the influence of label smoothing on a per dataset basis.

5 Conclusion

Most of KGs are confronted with complex relations modeling in the process of link prediction. Existing models learn less expressive features and hard to handle complex relations. Therefore, in this paper, we introduce a multi-layer convolutional network model M-DCN for link prediction. This model concatenates and reshapes the embeddings of entity and relation as a composition. Multi-scale convolution filters are utilized to learn embeddings characteristics of different aspects. Especially, the weights of these filters are related to the relation. Thus, an entity can output different features during the convolution process, which effectively distinguishes different entities and handles complex relations. During the model training, the 1-N scoring technique and Adam optimizer are utilized to speed up training. We adopt the dropout and label smoothing to lessen overfitting and improve generalization. Experimental results on five real-world datasets well demonstrate that M-DCN can effectively model complex relations and achieve the new state-of-the-art performance for triplet classification task on three datasets. Furthermore, we investigate the effects of different parameters on the model performance and perform further analysis. In future work, we plan to incorporate the relation paths to improve performance. KGs have multiple-step relation paths, such as (*Tom Cruise*, *Birth_in*, *New York*) and (*New York*, *State_in_country*, *United State*), which contain rich inference patterns between entities. Therefore, we can expand the current architecture by incorporating paths information. Moreover, KGs typically only contain correct triplets. Sampling useful incorrect training examples is a significant task. We further explore to generate incorrect triplets by utilizing the latest generative adversarial networks.

Acknowledgments

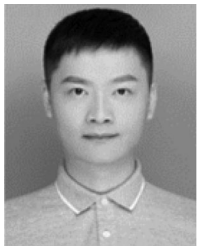
The authors Zhaoli Zhang and Zhifei Li contributed equally to this work. The authors sincerely thank anonymous reviewers for their constructive comments, which helped improve this paper. This work was supported in part by the National Natural Science Foundation of China under Grant 61875068, and Grant 61505064, and the Fundamental Research Funds for the Central Universities under Grant CCNU20ZT017 and Grant CCNU2020ZN008.

References

[1] H. Wang *et al.*, "Exploring high-order user preference on the knowledge graph for recommender systems," *ACM Trans. Inf. Syst.*, vol. 37, no. 3, pp. 32:1–32:26, 2019.
[2] Y. Cao, X. Wang, X. He, Z. Hu, and T. Chua, "Unifying knowledge graph learning and recommendation: Towards a better understanding of user preferences," in *Proc. 28th Int. Conf. World Wide Web*, 2019, pp. 151–161.
[3] H. Wang, F. Zhang, M. Zhao, W. Li, X. Xie, and M. Guo, "Multi-task feature learning for knowledge graph enhanced recommendation," in *Proc. 28th Int. Conf. World Wide Web*, 2019, pp. 2000–2010.
[4] Y. Hao, H. Liu, S. He, K. Liu, and J. Zhao, "Pattern-revising enhanced simple question answering over knowledge bases," in *Proc. 27th Int. Conf. Comput. Linguistics*, 2018, pp. 3272–3282.
[5] S. Hu, L. Zou, J. X. Yu, H. Wang, and D. Zhao, "Answering natural language questions by subgraph matching over knowledge graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 5, pp. 824–837, May 2018.
[6] H. Zhou, T. Young, M. Huang, H. Zhao, J. Xu, and X. Zhu, "Commonsense knowledge aware conversation generation with graph attention," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018, pp. 4623–4629.
[7] K. D. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor, "Freebase: A collaboratively created graph database for structuring human knowledge," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2008, pp. 1247–1250.
[8] G. A. Miller, "WordNet: A lexical database for english," *Commun. ACM*, vol. 38, no. 11, pp. 39–41, 1995.
[9] F. M. Suchanek, G. Kasneci, and G. Weikum, "Yago: A core of semantic knowledge," in *Proc. 16th Int. Conf. World Wide Web*, 2007, pp. 697–706.
[10] Q. Wang, Z. Mao, B. Wang, and L. Guo, "Knowledge graph embedding: A survey of approaches and applications," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 12, pp. 2724–2743, Dec. 2017.
[11] A. Bordes, N. Usunier, A. García-Durán, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *Proc. Advances Neural Inf. Process. Syst.*, 2013, pp. 2787–2795.
[12] Z. Wang, J. Zhang, J. Feng, and Z. Chen, "Knowledge graph embedding by translating on hyperplanes," in *Proc. 28th AAAI Conf. Artif. Intell.*, 2014, pp. 1112–1119.
[13] Y. Lin, Z. Liu, M. Sun, Y. Liu, and X. Zhu, "Learning entity and relation embeddings for knowledge graph completion," in *Proc. 29th AAAI Conf. Artif. Intell.*, 2015, pp. 2181–2187.

[14] G. Ji, S. He, L. Xu, K. Liu, and J. Zhao, "Knowledge graph embedding via dynamic mapping matrix," in *Proc. 53rd Annu. Meeting Assoc. Comput. Linguistics*, 2015, pp. 687–696.
[15] M. Nickel, V. Tresp, and H. Kriegel, "A three-way model for collective learning on multi-relational data," in *Proc. 28th Int. Conf. Mach. Learn.*, 2011, pp. 809–816.
[16] B. Yang, W. Yih, X. He, J. Gao, and L. Deng, "Embedding entities and relations for learning and inference in knowledge bases," in *Proc. 3rd Int. Conf. Learn. Representations*, 2015, pp. 1–12.
[17] T. Trouillon, J. Welbl, S. Riedel, É. Gaussier, and G. Bouchard, "Complex embeddings for simple link prediction," in *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 2071–2080.
[18] Y. Kim, "Convolutional neural networks for sentence classification," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2014, pp. 1746–1751.
[19] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances Neural Inf. Process. Syst.*, 2012, pp. 1106–1114.
[20] M. S. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *Proc. 15th Extended Semantic Web Conf.*, 2018, pp. 593–607.
[21] T. Dettmers, P. Minervini, P. Stenetorp, and S. Riedel, "Convolutional 2D knowledge graph embeddings," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 1811–1818.
[22] D. Q. Nguyen, T. D. Nguyen, D. Q. Nguyen, and D. Q. Phung, "A novel embedding model for knowledge base completion based on convolutional neural network," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, 2018, pp. 327–333.
[23] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Advances Neural Inf. Process. Syst.*, 2013, pp. 3111–3119.
[24] H. Xiao, M. Huang, and X. Zhu, "TransG: A generative model for knowledge graph embedding," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics*, 2016, pp. 2316–2325.
[25] T. Ebisu and R. Ichise, "Toruse: Knowledge graph embedding on a lie group," in *Proc. 30th AAAI Conf. Artif. Intell.*, 2018, pp. 1819–1826.
[26] T. Ebisu and R. Ichise, "Generalized translation-based embedding of knowledge graph," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 5, pp. 941–951, May 2020.
[27] H. Xiao, M. Huang, and X. Zhu, "From one point to a manifold: Knowledge graph embedding for precise link prediction," in *Proc. 25th Int. Joint Conf. Artif. Intell.*, 2016, pp. 1315–1321.
[28] S. Guo, Q. Wang, B. Wang, L. Wang, and L. Guo, "SSE: Semantically smooth embedding for knowledge graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 4, pp. 884–897, Apr. 2017.
[29] Z. Sun, Z. Deng, J. Nie, and J. Tang, "Rotate: Knowledge graph embedding by relational rotation in complex space," in *Proc. 7th Int. Conf. Learn. Representations*, 2019, pp. 1–18.
[30] Y. Lin, Z. Liu, H. Luan, M. Sun, S. Rao, and S. Liu, "Modeling relation paths for representation learning of knowledge bases," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2015, pp. 705–714.
[31] T. Liu, H. Liu, Y. Li, Z. Zhang, and S. Liu, "Efficient blind signal reconstruction with wavelet transforms regularization for educational robot infrared vision sensing," *IEEE-ASME Trans. Mechatron.*, vol. 24, no. 1, pp. 384–394, Feb. 2019.
[32] Y. Jia, Y. Wang, X. Jin, and X. Cheng, "Path-specific knowledge graph embedding," *Knowl.-Based Syst.*, vol. 151, pp. 37–44, 2018.
[33] Z. Wang and J. Li, "Text-enhanced representation learning for knowledge graph," in *Proc. 25th Int. Joint Conf. Artif. Intell.*, 2016, pp. 1293–1299.
[34] R. Xie, Z. Liu, J. Jia, H. Luan, and M. Sun, "Representation learning of knowledge graphs with entity descriptions," in *Proc. 30th AAAI Conf. Artif. Intell.*, 2016, pp. 2659–2665.
[35] N. Guan, D. Song, and L. Liao, "Knowledge graph embedding with concepts," *Knowl.-Based Syst.*, vol. 164, pp. 38–44, 2019.
[36] T. Liu, H. Liu, Y. Li, Z. Chen, Z. Zhang, and S. Liu, "Flexible FTIR spectral imaging enhancement for industrial robot infrared vision sensing," *IEEE Trans. Ind. Inform.*, vol. 16, no. 1, pp. 544–554, Jan. 2020.
[37] S. Guo, Q. Wang, L. Wang, B. Wang, and L. Guo, "Knowledge graph embedding with iterative guidance from soft rules," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 4816–4823.
[38] T. Liu, H. Liu, Z. Chen, and A. M. Lesgold, "Fast blind instrument function estimation method for industrial infrared spectrometers," *IEEE Trans. Ind. Inform.*, vol. 14, no. 12, pp. 5268–5277, Dec. 2018.
[39] D. Krompaß, M. Nickel, and V. Tresp, "Large-scale factorization of type-constrained multi-relational data," in *Proc. Int. Conf. Data Sci. Adv. Analytics*, 2014, pp. 18–24.
[40] D. Krompaß, S. Baier, and V. Tresp, "Type-constrained representation learning in knowledge graphs," in *Proc. 14th Int. Semantic Web Conf.*, 2015, pp. 640–655.
[41] M. Nickel, L. Rosasco, and T. A. Poggio, "Holographic embeddings of knowledge graphs," in *Proc. 30th AAAI Conf. Artif. Intell.*, 2016, pp. 1955–1961.
[42] H. Liu, Y. Wu, and Y. Yang, "Analogical inference for multi-relational embeddings," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 2168–2178.
[43] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Representations*, 2017, pp. 1–14.
[44] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. 5th Int. Conf. Artif. Intell. Statist.*, 2010, pp. 249–256.
[45] N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.
[46] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
[47] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2818–2826.
[48] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Representations*, 2015, pp. 1–15.
[49] K. Toutanova and D. Chen, "Observed versus latent features for knowledge base and text inference," in *Proc. 3rd Workshop Continuous Vector Space Models Their Compositionality*, 2015, pp. 57–66.
[50] A. Paszke *et al.*, "Automatic differentiation in pytorch," in *Proc. Advances Neural Inf. Process. Syst.*, 2017, pp. 1–4.

Zhaoli Zhang (Member, IEEE) received the MS degree in computer science from Central China Normal University, Wuhan, China, in 2004, and the PhD degree in computer science from the Huazhong University of Science and Technology, in 2008. He is currently a professor with the National Engineering Research Center for E-Learning, Central China Normal University. His research interests include knowledge services and natural language processing. He is a member of CCF (China Computer Federation).



Zhifei Li received the MS degree in collaborative & innovative center for educational technology from Central China Normal University, in 2018. He is currently working toward the PhD degree in the National Engineering Research Center for E-Learning, Central China Normal University. His current research interests include representation learning, graph neural networks, and knowledge graphs.

Hai Liu (Member, IEEE) received the MS degree in applied mathematics from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2010, and the PhD degree in pattern recognition and artificial intelligence from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2014. Since June 2017, he has been an assistant professor with the National Engineering Research Center for E-Learning, Central China Normal University, Wuhan. He was a "Hong Kong scholar" postdoctoral fellow with the Department of Mechanical Engineering, City University of Hong Kong, Kowloon, Hong Kong, where he was hosted by the professor You-Fu Li; he held the position two years till March 2019. He has authored more than 60 peer-reviewed articles in international journals from multiple domains such as pattern recognition, image processing, and knowledge graph embedding. More than six papers are selected as the ESI highly cited papers. His current research interests includes artificial intelligence, robot vision, big data processing, deep learning, knowledge graph, and pattern recognition. He has been frequently serving as a reviewer for more than six international journals including the *IEEE Transactions on Cybernetics*, *IEEE Transactions on Industrial Informatics* and *IEEE Transactions on Instrumentation and Measurement*. He is also a Communication Evaluation Expert for the National Natural Science Foundation of China.

Neal N. Xiong (Senior Member, IEEE) received the PhD degrees in sensor system engineering and in dependable sensor networks from Wuhan University, and the Japan Advanced Institute of Science and Technology, respectively. He is currently a professor with the Department of Mathematics and Computer Science, Northeastern State University, Tahlequah, OK. His research interests include cloud computing, machine learning, representation learning, and optimization theory. He is serving as the editor-in-chief, an associate editor, or an editor member for over ten international journals, including an associate editor for the *IEEE Transactions on Systems, Man, And Cybernetics: Systems and Information Science*.